CONTENTS

· Overview

1 Four kinds of intelligence, not a ladder

2 Humans: judgement, ambiguity and accountability

3 Rules and code: deterministic logic at scale

4 Where rules break down

5 Machine learning: patterns in structured data

6 What machine learning costs to keep running

7 Generative AI: unstructured input, flexible output

8 The real question: how much non-determinism can you tolerate?

9 Hybrid systems, and the heuristic

· Glossary

· Check yourself

· Where to go next

Advanced | System design decisions | Condensed | IBM Technology

Choosing the Right Kind of Intelligence: Humans, Rules, ML and Generative AI

Learn to match a problem to the cheapest system that can actually solve it, and to recognise when AI is the wrong tool.

For engineers and technical leads deciding where AI belongs in a system and where it does not · 26 July 2026

Every section, key point, term and question from the full lesson is here. The extended explanations and the additional examples are not.

Watch the source video | Read the full lesson | Read the highlights

BEFORE YOU START

- General software engineering experience
- A rough idea of what a trained ML model does
- Comfort reading Python

Overview

It is easy to fall into using AI for everything. Agent systems are genuinely powerful, and teams are wiring them across their stacks. But AI is not magic and it is not the best solution for every problem. Like any engineering decision, choosing whether to use it — and at what scale — is a trade-off across accuracy, cost, complexity and risk.

Most problems in software and data systems can be solved in one of four ways. Humans, for judgement and accountability. Rules or code, for deterministic logic. Machine learning, for statistical pattern recognition. Generative AI, for flexible reasoning over unstructured input. These are not a ladder you climb from human up to agent. They are distinct options, each with a specific role, and the skill is picking the right one.

The practical stakes are higher than they look. Most failures to reach production do not come from a bad model. They come from choosing the wrong kind of system at the start, and then spending months trying to make it behave like a different one.

KEY TAKEAWAYS

- ✓ Four options — humans, rules, ML, generative AI — are distinct tools, not rungs on a ladder.
- ✓ Humans are the right choice for high-stakes, ambiguous, accountability-bearing decisions, and they do not scale.
- ✓ Rules are the right choice when logic is known, explicit and stable, and when errors are unacceptable.
- ✓ Rules break when conditions change faster than you can update them, or when the logic grows too complex to write down.
- ✓ Machine learning fits when patterns exist in structured data but are not obvious enough to encode by hand.
- ✓ ML is not fire-and-forget: drift monitoring and retraining are permanent operating costs.
- ✓ Generative AI fits unstructured input, interpretation and synthesis — where flexibility matters more than exactness.
- ✓ The deciding question for generative AI is how much non-determinism you can tolerate, and the best systems combine all four.

01 Four kinds of intelligence, not a ladder

0:00

> Humans, rules, machine learning and generative AI are distinct options with distinct roles. Choosing among them is an ordinary engineering trade-off.

KEY POINTS

- → Four distinct options: humans, rules, ML, generative AI.
- → They are alternatives with different strengths, not stages of sophistication.
- → The trade-offs are accuracy, cost, complexity and risk — an ordinary engineering decision.
- → Reach for the cheapest deterministic option that works, and escalate only on a specific wall.
- → Replacing a working rule with a model buys non-determinism and cost for no gain.

The four options side by side

The row that decides most arguments is 'determinism'. Everything downstream — how you test, what you can guarantee, what happens on a bad day — follows from whether the same input always produces the same output.

	humans	rules	ML	generative AI
deterministic	no	yes	mostly	no
cost per call	very high	near zero	low	moderate to high
latency	minutes+	microsec	millisec	seconds
scales to	hundreds	unlimited	millions	limited by cost
interpretable	explains	fully	varies	post-hoc only
testable	no	exhaustive	statistical	hard
handles novelty	yes	no	poorly	yes
accountability	yes	designer	designer	designer

02 Humans: judgement, ambiguity and accountability

1:27

> People handle incomplete and conflicting information, apply ethics and context, and can be held responsible. They are also expensive and do not scale.

KEY POINTS

- → Humans handle ambiguity, conflicting information, context and ethics.
- → Use them for high-stakes decisions, liability, and genuine ambiguity.
- → Accountability is not an engineering property — no accuracy improvement creates it.
- → The cost is expense, latency and an inability to scale.
- → The usual answer is a split: automate the filtering, keep the judgement.

Splitting a decision by what it actually requires

Hiring, taken apart. The mechanical parts are automated and the judgement is preserved — and notice the automated stage is deliberately built to over-include, because a false negative here is invisible and unrecoverable.

```
def screen(applications):
    # rules: mechanical, auditable, and deliberately generous
    eligible = [a for a in applications
                if a.work_authorised and a.years_experience >= 2]

    # ML: rank by similarity to past successful hires — a suggestion, not a gate
    ranked = model.rank(eligible)

    # humans: everything that carries consequence
    return ranked[:40]          # a person reads all forty

# the filter's job is to not lose good candidates, not to pick the winner
```

03 Rules and code: deterministic logic at scale

3:01

> When the logic is known, stable and must not be wrong, ordinary code is fast, cheap, testable and completely interpretable.

KEY POINTS

- → Rules suit known, explicit, stable logic where errors are unacceptable.
- → Payment processing, validation, data transformation, access control.
- → Code is fast, cheap, interpretable, debuggable and exhaustively testable.
- → A tested rule behaves identically forever; a model offers only a statistical claim.
- → Anything that must never be wrong should not be probabilistic.

What deterministic actually buys you

Four assertions and the behaviour is pinned for the lifetime of the code. No equivalent exists for a model: you can measure accuracy on a sample, but you cannot assert on every input.

```
def authorise(account, amount):
    if amount <= 0:                  return DECLINE("invalid amount")
    if account.frozen:               return DECLINE("account frozen")
    if amount > account.available:   return DECLINE("insufficient funds")
    return APPROVE


def test_authorise():
    assert authorise(Account(available=100), 50)  == APPROVE
    assert authorise(Account(available=100), 150) == DECLINE
    assert authorise(Account(available=100), 0)   == DECLINE
    assert authorise(Account(frozen=True), 1)     == DECLINE

# every branch covered. behaviour fixed forever. no drift, no monitoring,
# no retraining, no confidence interval.
```

04 Where rules break down

3:44

> Rules fail when conditions change constantly or the logic becomes too complex to state. Fraud detection is the standard example.

KEY POINTS

- → Adaptive adversaries make any fixed threshold decay from the moment it ships.
- → Combinatorial complexity makes correct logic impossible to state, with no adversary involved.
- → The symptom is the same: exception piled on exception, and fear of changing the code.
- → Adding another special case treats the symptom rather than the cause.
- → At the wall, the question changes from 'what is the rule?' to 'what distinguishes these cases?'

A rule set decaying in public

Each line was added after a real incident and was correct at the time. Together they encode a history of past attacks rather than a model of fraud — which is why they catch last quarter's patterns and miss this quarter's.

```
def is_fraud(txn):
    if txn.amount > 1000:                       return True   # week 1
    if txn.amount > 900 and txn.foreign:        return True   # week 3
    if 995 < txn.amount < 1000:                 return True   # week 5
    if txn.count_last_hour > 4:                 return True   # week 6
    if txn.count_last_hour > 2 and txn.night:   return True   # week 8
    if txn.amount < 5 and txn.new_merchant:     return True   # week 9
    # ... 340 more lines, each from an incident review

# every rule was right when written. attackers now transact at 994.
# nobody on the team will touch this file.
```

05 Machine learning: patterns in structured data

4:29

> Learn the distinguishing pattern from examples instead of stating it. Right when patterns exist but are not obvious enough to encode by hand.

KEY POINTS

- → ML suits structured data where patterns exist but are not obvious enough to state.
- → You supply labelled examples; the model derives the logic.
- → Standard applications: fraud, churn, demand forecasting, recommendations.
- → The work moves from branching logic to feature engineering and label quality.
- → Labels inherited from an old rule system teach the model that system's blind spots.

Rules versus a learned model on the same problem

The right-hand version never mentions a threshold. It learns which combinations of features separate the classes, including interactions nobody would think to write — and it is retrained rather than edited when behaviour shifts.

```
# rules: you state the logic
if txn.amount > 1000 and txn.foreign and txn.night:
    flag()

# ML: you supply examples and features, and the logic is derived
features = [
    "amount", "amount_vs_user_median", "hour_of_day",
    "seconds_since_last_txn", "merchant_risk_score",
    "distance_from_last_txn_km", "is_new_merchant_for_user",
]

model = GradientBoostingClassifier()
model.fit(X_train[features], y_train)      # y = labelled fraud/not fraud

score = model.predict_proba(txn_features)[0][1]    # 0.0 to 1.0
```

06 What machine learning costs to keep running

5:13

> Models scale well and behave predictably, and they need monitoring, retraining and an honest account of what they can explain.

KEY POINTS

- → Models scale well and behave predictably, but require ongoing monitoring.
- → Drift degrades accuracy silently, with no error and no code change.
- → Data drift is a change in the inputs; concept drift is a change in the input-outcome relationship.
- → Concept drift needs outcomes to detect, and outcomes often arrive late.
- → Feature attribution explains which inputs drove a prediction, never why the world behaves that way.

Detecting data drift

Population Stability Index compares the live feature distribution against the training baseline. Run it per feature on a schedule; the conventional reading is that below 0.1 is stable, 0.1 to 0.2 warrants attention, and above 0.2 means retrain.

```
import numpy as np

def psi(baseline, current, bins=10):
    edges = np.percentile(baseline, np.linspace(0, 100, bins + 1))
    edges[0], edges[-1] = -np.inf, np.inf

    b = np.histogram(baseline, bins=edges)[0] / len(baseline)
    c = np.histogram(current,  bins=edges)[0] / len(current)

    b, c = np.clip(b, 1e-6, None), np.clip(c, 1e-6, None)
    return float(np.sum((c - b) * np.log(c / b)))

for feature in FEATURES:
    score = psi(train_df[feature], live_df[feature])
    if score > 0.2:
        alert(f"{feature} drifted: PSI={score:.3f}")

# catches an upstream unit change on the day it happens, rather than
# three weeks later when someone notices revenue is down
```

07 Generative AI: unstructured input, flexible output

5:53

> Right when inputs are text or media, tasks need interpretation or transformation, and flexibility matters more than precision.

KEY POINTS

- → Generative AI suits unstructured input: text, documents, media.
- → It suits interpretation, synthesis and transformation rather than numeric prediction.
- → Use it where flexibility matters more than precision and some error is tolerable.
- → The distinguishing property is that the input has no schema.
- → Time to first result is very fast, which conceals how much work reliability will take.

Why rules cannot take this input

One support message containing three intents, a factual error, an implied deadline and an emotional register that should change the reply. There is no schema here, and every attempt to impose one loses something a human reader would act on.

```
"hi so we upgraded to the enterprise plan last tuesday i think? but the
sso thing still isnt working for half the team and my colleague said
something about a separate config?? also we were told this would include
priority support but nobody has replied in 3 days. we have an audit
friday so this is getting urgent"

# intents      : SSO fault, billing/entitlement query, support SLA complaint
# facts to check: upgrade date, whether SSO needs separate configuration
# implicit     : hard deadline on Friday
# register     : frustrated, escalating

# no parser recovers this. an LLM reads it the way a person does.
```

08 The real question: how much non-determinism can you tolerate?

7:21

> Generative systems do not guarantee correctness across runs, which makes them hard to test and costly at scale. That tolerance is the deciding criterion.

KEY POINTS

- → The same input can produce different outputs, so exact-match testing does not apply.
- → Correctness is not guaranteed across runs, and costs are higher at scale.
- → Replace assertions with property checks, pass rates over repeated runs, golden sets and judge models.
- → You are measuring a distribution and judging its tail, not verifying correctness.
- → Arithmetic and aggregation belong in a tool, not in the model's head.

Testing something that will not sit still

Three assertions that survive non-determinism. Note the pass-rate test: running once tells you almost nothing, and a case that passes eight times in ten is a very different thing from one that passes always.

```
def test_extraction_properties():
    out = extract(MESSAGE)
    assert set(out.intents) <= VALID_INTENTS       # never invents a category
    assert out.deadline is None or is_iso_date(out.deadline)
    assert out.urgency in ("low", "medium", "high")


def test_pass_rate(n=20, threshold=0.9):
    passes = sum("sso_fault" in extract(MESSAGE).intents for _ in range(n))
    assert passes / n >= threshold, f"only {passes}/{n}"


def test_golden_set():
    scores = [grade(model_answer(c.input), c.expected) for c in GOLDEN]
    assert mean(scores) >= BASELINE - 0.02          # regression guard
```

09 Hybrid systems, and the heuristic

8:11

> The best systems use all four in combination. 'Make everything agents' was never meant literally.

KEY POINTS

- → The strongest systems combine all four, each on the part it fits.
- → Apply the choice per component, not per product.
- → Humans for judgement and accountability; rules for stable explicit logic.
- → ML for patterns in past structured data; generative AI for unstructured interpretation.
- → Most production failures come from choosing the wrong system, not from a bad model.

One feature, four technologies

What 'we built AI fraud detection' actually contains. Each stage uses the cheapest system that can do its job, and the expensive, non-deterministic parts are kept off the path where correctness is required.

```
incoming transaction
     │
     ├─ rules      hard constraints: frozen account, invalid amount,
     │             sanctions list          → decline, deterministic
     │
     ├─ ML         risk score from behavioural features   → 0.0–1.0
     │
     ├─ rules      thresholds: >0.95 block, >0.60 review, else approve
     │             (a business decision, in reviewable code)
     │
     ├─ humans     analysts work the review queue, and their verdicts
     │             become next month's training labels
     │
     └─ generative writes the customer-facing explanation, and drafts
                   the analyst's case summary from the evidence
```

Glossary

Deterministic system

One where identical input always produces identical output. Exhaustively testable, and its behaviour is fixed once verified.

Rule-based system

Explicit conditional logic written by a person. Fast, cheap, interpretable and correct only for the cases anticipated.

Machine learning

Deriving a predictive function from labelled examples rather than stating it. Suits structured data with patterns too complex to encode by hand.

Model drift

Gradual degradation in accuracy as the world diverges from the training data. Silent: no error, no code change.

Data drift

A change in the distribution of incoming features relative to training. Detectable by comparing distributions.

Concept drift

A change in the relationship between inputs and outcomes. Harder to detect, because it requires outcomes that often arrive late.

Population Stability Index (PSI)

A measure of how much a feature's distribution has shifted from a baseline. Commonly read as stable below 0.1 and needing retraining above 0.2.

Non-determinism

The property that identical input may produce different output across runs. The defining cost of generative systems.

Golden set

A fixed collection of inputs with known-good outputs, scored repeatedly to catch regressions in a system you cannot assert on exactly.

LLM as judge

Using a second model to score an output against a rubric, where the output is too open-ended for a programmatic check.

Feature attribution

Identifying which inputs most influenced a prediction. Explains the model's behaviour, never the underlying causality.

Hybrid system

An architecture combining humans, rules, ML and generative AI, each applied to the part of the problem it fits.

Check yourself

Answer first, then open each one.

Why is 'accountability' a different kind of selection criterion from accuracy, cost or latency?

The others are measurable properties of the system that engineering effort can improve. Accountability is not a property of the system at all — it is an arrangement about who answers for the outcome, and no amount of model improvement produces it. A system that is right 99% of the time still cannot be responsible for the remaining 1%.

A team replaces a tested, working validation rule with a fine-tuned classifier. What have they traded away, and what have they gained?

They have given up exhaustive testability, deterministic behaviour, near-zero cost and full interpretability, and taken on drift monitoring and retraining. They gained nothing, because the logic was already known, explicit and stable — which is exactly the case rules are best at. This is the characteristic error of treating the four options as a ladder.

A fraud model scores 0.97 AUC in evaluation and performs poorly in production. What is the most likely cause, and why does evaluation not reveal it?

The training labels came from cases the old rule system flagged, so the model learned to reproduce those rules including their blind spots. Evaluation does not reveal it because the test set carries the identical bias — fraud the rules never caught was never labelled and appears in neither split. The fix is to label a random sample independently of what the rules did.

Why is concept drift harder to detect than data drift?

Data drift is a change in the input distribution, which you can detect immediately by comparing incoming features against the training baseline. Concept drift is a change in the input-to-outcome relationship, so detecting it requires knowing the outcomes — and those often arrive weeks later. Churn is only observable once the customer has already gone.

Why should an agent asked to analyse a year of transactions never compute the totals itself?

Because it produces a plausible number rather than a computed one, and a plausible total is indistinguishable from a correct one at a glance. Aggregation is exactly the deterministic work code guarantees, so the model should decide what to compute and describe the result while a tool produces the figures.

Why is 'should this feature use AI?' usually the wrong question?

Because a feature contains many decisions and they rarely want the same technology. Fraud detection alone wants rules for hard constraints, ML for scoring, rules again for thresholds, humans for appeals and generative AI for the customer notification. The choice belongs at the component level, and asking at the product level forces one answer onto four different problems.

Where to go next

- → Take one feature you already ship and decompose it into components, labelling each as human, rule, ML or generative — most teams find at least one component using a heavier technology than it needs.
- → Implement the PSI check over your production model's features and backfill it against the last six months of data to see whether drift you never noticed was already happening.
- → Take a generative feature with no tests and write the three test shapes from this lesson — property assertions, a pass rate over repeated runs, and a golden set with a regression threshold.
- → Audit any rule file that has grown past a few hundred lines against the wall checklist, and see whether the answer is a learned model or simply a rewrite.
- → For a fraud or abuse model you own, check where the training labels came from; if they came from the previous rule system, estimate what fraction of real cases could never have been labelled.
- → Find a place where an LLM is doing arithmetic or aggregation and replace it with a tool call, then compare outputs on a hundred cases — the error rate is usually higher than anyone expected.
